

## Lista Teórica 06 — Gabarito

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula 06 — Pré-processamento e Pipelines

**Exercício 1 — a pergunta que separa grave de leve.** O critério da Aula 06 para classificar um vazamento não é “quanto a etapa aprende dos dados”, e sim uma pergunta só: **a etapa olha o  $Y$ ?**

Classifique cada situação abaixo como vazamento **grave**, **leve** ou **nenhum**, e justifique em uma linha aplicando esse critério.

- Padronizar todas as covariáveis usando a média e o desvio-padrão calculados no conjunto *completo*, antes de separar treino e teste.
- Selecionar, entre 10 000 covariáveis, as 50 mais correlacionadas com  $Y$  usando todos os dados, e só então rodar a validação cruzada com essas 50.
- Ajustar o PCA no conjunto completo, reduzir a 20 componentes, e só então separar treino e teste.
- Imputar valores faltantes pela *mediana da coluna*, calculada no conjunto completo.
- Num banco em que cada paciente aparece em várias linhas (uma por consulta), fazer KFold sobre as linhas.
- Padronizar dentro de um Pipeline, com GridSearchCV por fora.

### Solução

|     | classificação | por quê                                                                                                                                                   |
|-----|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| (a) | <b>leve</b>   | O <code>StandardScaler</code> calcula média e desvio das colunas de $\mathbf{X}$ ; nunca vê $Y$ . Não há como fabricar sinal a partir de ruído.           |
| (b) | <b>grave</b>  | A seleção é feita <i>pela correlação com <math>Y</math></i> . Cada dobra de validação já contribuiu para escolher as covariáveis que serão testadas nela. |
| (c) | <b>leve</b>   | O PCA aprende direções de <i>todas</i> as colunas ao mesmo tempo, o que parece grave — mas ele só olha $\mathbf{X}$ .                                     |
| (d) | <b>leve</b>   | A mediana é calculada na coluna de $\mathbf{X}$ . Mesmo argumento de (a).                                                                                 |
| (e) | <b>grave</b>  | Não é sobre pré-processamento: é a <i>mesma unidade</i> nos dois lados da divisão. O modelo vê consultas do mesmo paciente no treino e na validação.      |
| (f) | <b>nenhum</b> | É exatamente a conduta correta: a padronização é reajustada dentro de cada dobra.                                                                         |

Duas observações que a tabela não cabe.

A primeira é sobre (c): ele é o caso em que a intuição mais engana. Ajustar o PCA no conjunto todo *de fato* usa informação do teste — as componentes são outras. Mas “usar

informação” não é o mesmo que “inflar o desempenho medido”, e sem olhar o  $Y$  não há como fabricar acerto. A Aula E2 mede isso.

A segunda é sobre (e): **nenhum Pipeline protege desse caso**. O Pipeline garante que cada etapa seja ajustada só no treino da dobra, mas não tem como saber que duas linhas são o mesmo paciente. A ferramenta é o `GroupKFold`, passando o identificador do paciente em `groups`. Vale guardar o par: *o Pipeline resolve (a), (c) e (d); só o desenho da divisão resolve (e)*.

**Exercício 2 — por que um custa 0,40 e o outro custa nada.** A nota traz duas medições. Com  $\mathbf{X}$  e  $y$  gerados **independentemente** (isto é,  $y$  é ruído puro, e o  $R^2$  verdadeiro de qualquer modelo é 0):

- selecionar as covariáveis mais correlacionadas com  $y$  antes da validação cruzada produz  $R^2 = +0,40$ ;
  - padronizar fora da dobra, num problema com sinal de verdade e escalas muito díspares, produz 0,8466 — contra 0,8466 fazendo certo.
- (a) Explique o mecanismo do primeiro número. Se  $y$  é ruído puro, de onde sai um  $R^2$  positivo?
- (b) Suponha  $p$  covariáveis independentes de  $y$ , e que você fique com as  $m$  de maior correlação amostral com  $y$ . O que acontece com o  $R^2$  inflado quando  $p$  cresce, com  $n$  e  $m$  fixos? Por quê?
- (c) Explique por que a padronização não consegue produzir o mesmo efeito, por mais que ela use o conjunto todo.
- (d) Se o efeito da padronização é desprezível, por que ainda assim se recomenda pô-la dentro do Pipeline?

### Solução

(a) Com  $y$  de ruído puro, a correlação *populacional* entre qualquer covariável e  $y$  é zero — mas a correlação *amostral* não é: ela flutua em torno de zero com desvio da ordem de  $1/\sqrt{n}$ . Ao ficar com as covariáveis de maior correlação amostral, você está escolhendo, entre milhares de flutuações, as que por acaso ficaram maiores. Não é um subconjunto qualquer: é o subconjunto que melhor se ajusta *a este  $y$  específico*, inclusive à parte dele que aparece nas dobras de validação.

Quando a validação cruzada roda depois, cada dobra de validação já participou da escolha das covariáveis que serão testadas nela. O ruído daquela dobra já está codificado na lista de variáveis — e o modelo o reencontra. É o mesmo mecanismo do Exercício 4 da Lista Teórica 03 (selecionar é otimizar), aqui aplicado a covariáveis em vez de hiperparâmetros.

(b) O  $R^2$  inflado **aumenta** com  $p$ . Quanto mais covariáveis você examina, maior é o máximo das  $p$  correlações amostrais — é o mesmo argumento do máximo de  $M$  variáveis aleatórias do Exercício 4 da Lista 03. Com  $p = 10$  você escolhe as  $m$  melhores de dez flutuações; com  $p = 10\,000$ , as  $m$  melhores de dez mil, que são muito maiores.

Isso é desconfortável, porque a situação em que a seleção de variáveis parece mais necessária —  $p$  enorme — é exatamente aquela em que fazê-la errado custa mais caro.

(c) Porque o `StandardScaler` **nunca vê o  $y$** . Ele calcula duas estatísticas por coluna de  $\mathbf{X}$ : a média e o desvio-padrão. Usar o conjunto completo em vez de só o treino muda essas duas estatísticas por uma quantidade da ordem de  $1/\sqrt{n}$ , e o efeito é o mesmo para todas as observações, independentemente do valor de  $y$  delas.

Dito de outro modo: a etapa não tem nenhum canal por onde a informação sobre a resposta possa entrar. Ela pode piorar ou melhorar o condicionamento numérico, mas não pode fazer um modelo parecer melhor do que é. Foi essa a medição: 0,8466 contra 0,8466.

(d) Três razões, e nenhuma delas é o vazamento:

- (i) **Custa uma linha.** Se a conduta certa é gratuita, não há motivo para debater se a errada é aceitável neste caso.
- (ii) **A conduta é a mesma para todas as etapas.** Quem se acostuma a padronizar fora do *pipeline* vai, mais cedo ou mais tarde, selecionar variáveis fora dele — e aí custa 0,40. A disciplina vale mais que o caso isolado.
- (iii) **Reprodutibilidade em produção.** O Pipeline carrega a média e o desvio aprendidos no treino, e os aplica automaticamente a qualquer observação nova. Sem ele, alguém precisa lembrar de guardar e reaplicar esses números — e é aí que aparecem erros silenciosos.

O que a medição muda não é a conduta; é **onde gastar vigilância**. Numa revisão de código, a pergunta cara é “como as covariáveis foram escolhidas?” e “as dobras respeitam as unidades?”, não “o `StandardScaler` está no lugar certo?”.

**Exercício 3 — escrevendo o pipeline.** Um banco tem as seguintes colunas:

- `idade`, `renda`, `saldo` — numéricas, com valores faltantes em `renda`;
- `estado`, `profissao` — categóricas, sem faltantes;
- `inadimplente` — a resposta, binária.

A receita desejada é: imputar as numéricas pela mediana, padronizá-las, codificar as categóricas em variáveis indicadoras, e ajustar uma regressão logística com penalização  $\ell_2$ , escolhendo o  $C$  por validação cruzada de 5 dobras.

- (a) Escreva o código, usando `ColumnTransformer`, `Pipeline` e `GridSearchCV`.
- (b) Na sua solução, quantas vezes a mediana de `renda` é calculada ao longo de toda a busca, se a grade de  $C$  tem 6 valores? Justifique.
- (c) Por que a ordem imputar  $\rightarrow$  padronizar importa, e não o contrário?

### Solução

(a)

```
1 import numpy as np
2 import sklearn.model_selection as skm
3 from sklearn.compose import ColumnTransformer
4 from sklearn.impute import SimpleImputer
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.pipeline import Pipeline
7 from sklearn.preprocessing import OneHotEncoder, StandardScaler
8
9 numericas = ["idade", "renda", "saldo"]
10 categoricas = ["estado", "profissao"]
11
12 prep = ColumnTransformer([
```

```

33     ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
34                       ("escala", StandardScaler())]), numericas),
35     ("cat", OneHotEncoder(handle_unknown="ignore"), categoricas),
36 ])
37
38 tubo = Pipeline([("prep", prep),
39                  ("modelo", LogisticRegression(penalty="l2", max_iter=2000))])
40
41 busca = skm.GridSearchCV(
42     tubo,
43     {"modelo__C": np.logspace(-3, 2, 6)},
44     cv=skm.StratifiedKFold(5, shuffle=True, random_state=0),
45     scoring="roc_auc",
46 ).fit(X, y)

```

Dois detalhes que valem nota: `handle_unknown="ignore"` evita que o ajuste quebre quando uma categoria aparece só na validação, e `StratifiedKFold` mantém a proporção de inadimplentes em cada dobra — em classificação é o padrão do `GridSearchCV`, mas explicitá-lo deixa o `shuffle` sob controle (Exercício 3 da Lista prática 03).

(b) **30 vezes:** 6 valores de  $C \times 5$  dobras. Em cada combinação, o `GridSearchCV` chama `fit` do *pipeline inteiro* usando só as 4/5 de treino daquela dobra, e o `SimpleImputer` recalcula a mediana ali. (Com `refit=True`, que é o padrão, há ainda um 31º ajuste no conjunto completo ao final, para produzir o `best_estimator_`.)

Isso é o oposto de imputar uma vez, no começo, no conjunto todo. Redundante? Sim, computacionalmente. Mas é o que torna a estimativa honesta — e note que a mediana muda de dobra para dobra, o que é justamente a variabilidade que a validação cruzada precisa capturar.

(c) Porque a padronização usa média e desvio-padrão, e ambos são afetados pelos valores faltantes. Se você padronizar primeiro, o `StandardScaler` encontra NaN na coluna `renda` e propaga NaN para todas as estatísticas — na melhor das hipóteses o código quebra, na pior ele devolve uma coluna inteira de NaN.

Mesmo que não quebrasse, a ordem inversa seria conceitualmente errada: imputar depois de padronizar significaria inserir a mediana *da escala padronizada*, o que muda a média da coluna e desfaz a padronização que acabou de ser feita. Imputar primeiro deixa a coluna completa, e aí a padronização vê os dados que efetivamente irão para o modelo.

**Exercício 4 — a divisão que nenhum pipeline conserta.** Um laboratório mede a expressão de 20 000 genes em 60 pacientes, três amostras por paciente (180 linhas), e quer prever se o tumor responde ao tratamento. Um analista monta o `Pipeline` correto — imputação e padronização dentro dele — e reporta acurácia de 0,95 por validação cruzada de 10 dobras sobre as 180 linhas.

- Qual é o problema, e por que o `Pipeline` não o resolve?
- Qual é o número aproximado de pacientes que aparecem *ao mesmo tempo* no treino e na validação de uma dobra típica? Justifique.
- Como corrigir? Escreva a chamada.
- O laboratório também quer selecionar os 200 genes mais associados à resposta. Onde essa etapa tem de ficar?

**Solução**

(a) As três amostras de um mesmo paciente são **a mesma unidade experimental**. Ao dividir por linha, quase todo paciente terá amostras no treino e amostras na validação, e o modelo não precisa aprender nada sobre biologia: basta reconhecer o perfil de expressão *daquele paciente*, que ele já viu.

O **Pipeline** não resolve porque ele controla *quando* cada etapa é ajustada, não *como* as observações são repartidas. Ele garante que a padronização use só o treino da dobra — e o treino da dobra já está contaminado. É a distinção do Exercício 1: o **Pipeline** resolve os itens (a), (c) e (d); este é o item (e).

(b) Praticamente **todos os 60**. Numa dobra de 10 dobras, cerca de 18 das 180 linhas vão para a validação. Cada uma pertence a um paciente que tem outras duas linhas, e a chance de as outras duas caírem também na mesma dobra é pequena — a probabilidade de um paciente ter as três amostras na mesma dobra de 1/10 é da ordem de  $(1/10)^2 = 1\%$ . Logo, quase todos os pacientes representados na validação também estão no treino, e como são 18 linhas de até 18 pacientes distintos, o vazamento atinge quase toda a dobra.

(c) Dividindo por **grupo**, com o identificador do paciente:

```
1 import sklearn.model_selection as skm
2
3 cv = skm.GroupKFold(n_splits=10)
4 notas = skm.cross_val_score(tubo, X, y, cv=cv, groups=id_paciente,
5                             scoring="accuracy")
```

O **GroupKFold** garante que todas as linhas de um paciente fiquem do mesmo lado da divisão. A acurácia vai cair — e o número menor é o verdadeiro.

(d) **Dentro do Pipeline**, como um passo (**SelectKBest**, por exemplo), de modo que seja reajustada em cada dobra. Este é o item (b) do Exercício 1, e é o vazamento de  $R^2 = +0,40$  — agravado aqui pela proporção: 20 000 genes para 60 pacientes.

Note que o problema deste exercício exige as **duas** correções ao mesmo tempo, e elas são independentes. Fazer só o **GroupKFold** deixa a seleção de genes vazando; fazer só a seleção dentro do *pipeline* deixa o mesmo paciente nos dois lados. É por isso que vale ter as duas perguntas na cabeça separadamente: *a etapa olha o Y?* e *a divisão respeita a unidade?*